

Comparison of Pick-and-Place Programming Approaches for mycobot_320 M5 in ROS2 / Gazebo Harmonic

Alexander Todorov · PhD Research · Technical University of Sofia · May 2026

ROS2 Humble · Gazebo Harmonic (gz-sim 8) · ros2_control · Ubuntu 22.04

Abstract

This paper presents and compares two programmatic approaches for implementing pick-and-place manipulation with the mycobot_320 M5 six-degree-of-freedom robot arm in a Gazebo Harmonic physics simulation.

Approach A

uses manually determined, hardcoded joint angles derived iteratively via the rqt Joint Trajectory Controller plugin.

Approach B

uses analytical Inverse Kinematics (IK) solved numerically by the ikpy library from the robot's URDF kinematic chain, allowing Cartesian waypoint specification. Both approaches are implemented as self-contained ROS2 Humble packages. We describe the methodology, calculations, code, experimental results, and limitations of each approach, and provide a structured comparison to guide practitioners in selecting an appropriate method.

1. Introduction

Pick-and-place is a fundamental manipulation task in robotics. Programming a robot arm to perform this task requires specifying target configurations in either *joint space* (joint angles) or *Cartesian space* (XYZ coordinates). The two representations lead to fundamentally different programming workflows with different trade-offs.

This work uses the **mycobot_320 M5 (2022)** — a 6-DOF serial robot arm by Elephant Robotics — running in a **Gazebo Harmonic** physics simulation under **ROS2 Humble** on Ubuntu 22.04. The robot is equipped with an adaptive parallel gripper. A foam cube (40×60×60 mm, 10 g) serves as the manipulation target.

The goal of this comparison is to understand the engineering trade-offs between manual joint-space specification (Approach A) and automated Cartesian-space IK solving (Approach B), with particular attention to implementation complexity, reliability, and extensibility.

2. System Description

2.1 Robot: mycobot_320 M5

Property	Value
Degrees of Freedom	6 (revolute joints)
Payload	1 kg (real hardware)
Reach	~320 mm
Gripper	Adaptive parallel jaw (4-finger)
Simulation	Gazebo Harmonic, gz-sim 8
Control	ros2_control: JointTrajectoryController + JointGroupPositionController

2.2 Link Lengths (from URDF)

j1 base offset: 173.9 mm j1→j2: 88.78 mm j2→j3: 135 mm j3→j4: 120 mm j4→j5: 95 mm j5→j6: 65.5 mm j6→gripper center: ~144 mm

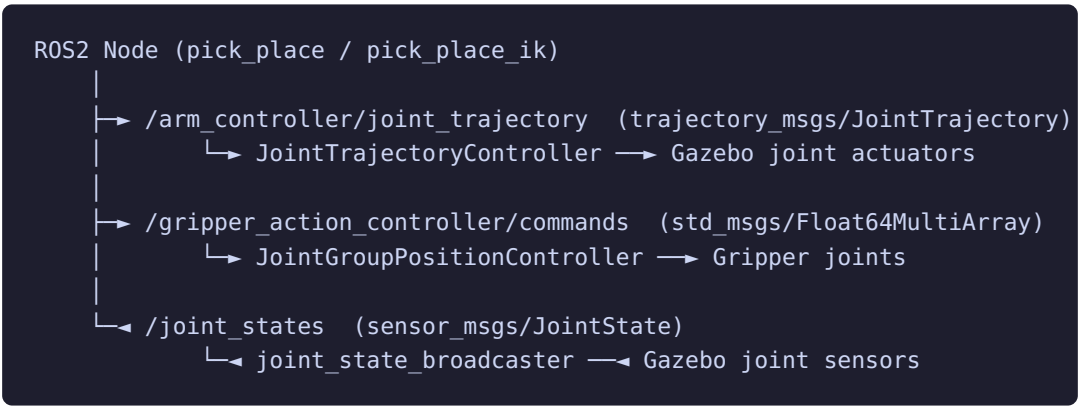
2.3 Joint Naming Convention

ROS2 Joint Name	Physical Joint	Motion
joint2_to_joint1	J1 (base)	Rotate left/right (yaw)
joint3_to_joint2	J2 (shoulder)	Pitch forward/back
joint4_to_joint3	J3 (elbow)	Pitch forward/back
joint5_to_joint4	J4 (wrist 1)	Pitch
joint6_to_joint5	J5 (wrist 2)	Rotate (roll)
joint6output_to_joint6	J6 (wrist 3)	Rotate (yaw)

2.4 Manipulation Target: Cube

Property	Value
Dimensions	40 mm (X) × 60 mm (Y) × 60 mm (Z)
Mass	10 g
Friction coefficient	$\mu = 50$ (simulation)
Position in world	X=0.262 m, Y=-0.0842 m, Z=0.030 m
Yaw orientation	90° (1.5708 rad)

2.5 Control Architecture



3. Approach A — Hardcoded Joint Angles

3.1 Overview

In Approach A, every robot waypoint is specified as a fixed list of six joint angles (in radians). These angles were determined *manually* using the **rqt Joint Trajectory Controller** plugin: sliders were adjusted until the arm visually aligned with the desired position, and the resulting joint values were recorded. The process was repeated iteratively until the gripper reliably grasped the cube.

3.2 Waypoints

Waypoint	J1	J2	J3	J4	J5	J6	Purpose
HOME	0.0004	0.0042	-0.0228	0.0045	0.0527	0.0077	Safe starting/

							ending position
GRIPPER_READY	0.0	-0.0006	-0.0856	0.0226	1.5567	0.0	Arm extended forward, gripper vertical
PRE_GRASP	0.0	-0.7050	-0.7237	-0.1689	1.5772	0.0	Above cube, aligned for descent
GRASP	0.0	-0.8150	-0.8340	-0.1689	1.5772	0.0	Gripper around cube

LIFT re-uses PRE_GRASP angles (same XY, higher Z by joint geometry).

3.3 Methodology — How Waypoints Were Found

The manual calibration procedure:

1. Launch Gazebo simulation with Approach A launch file
2. Open rqt → Plugins → Robot Tools → Joint Trajectory Controller
3. Activate `arm_controller` in the plugin
4. Adjust sliders for each joint while watching the arm in Gazebo
5. When visually aligned, record the 6 joint values
6. Copy values into the Python source code as constants
7. Run the pick sequence, observe where it fails, adjust and repeat

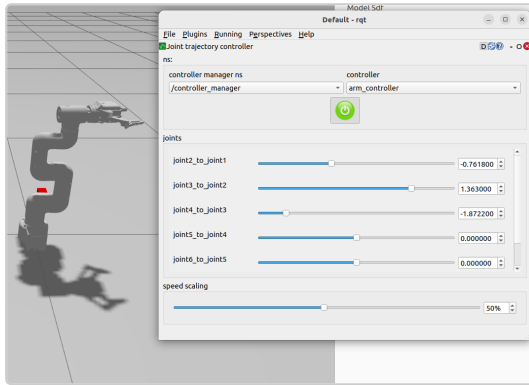


Fig. A1 — rqt Joint Trajectory Controller: manually adjusting joint angles

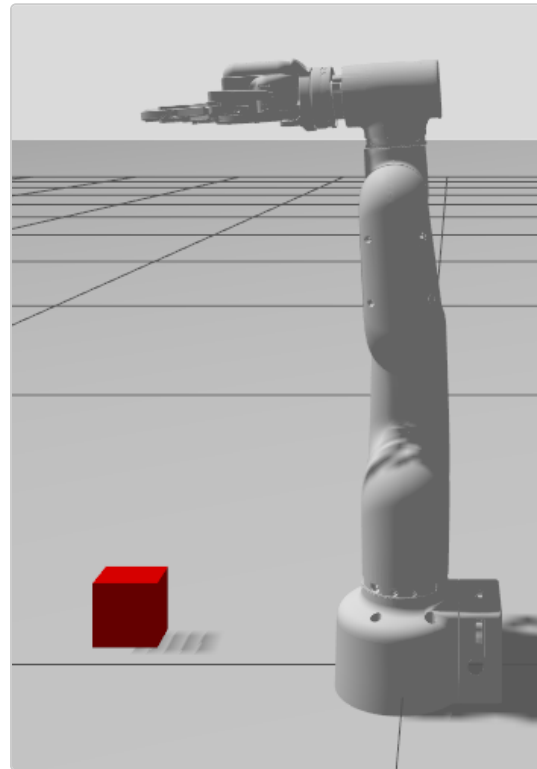


Fig. A2 — Gripper-ready pose (arm extended, gripper vertical)

3.4 Angle Calculation (radians ↔ degrees)

$$\text{angle_deg} = \text{angle_rad} \times (180 / \pi) \quad \text{angle_rad} = \text{angle_deg} \times (\pi / 180)$$

Example PRE_GRASP J2: $-0.7050 \text{ rad} = -0.7050 \times 57.296^\circ = -40.4^\circ$

Example PRE_GRASP J3: $-0.7237 \text{ rad} = -41.5^\circ$

Example GRASP J2: $-0.8150 \text{ rad} = -46.7^\circ$ (6.3° deeper than PRE_GRASP)

3.5 Code — Key Implementation

```
ARM_JOINTS = ['joint2_to_joint1', 'joint3_to_joint2', 'joint4_to_joint3',
              'joint5_to_joint4', 'joint6_to_joint5', 'joint6output_to_joint6']

HOME          = [+0.0004, +0.0042, -0.0228, +0.0045, +0.0527, +0.0077]
GRIPPER_READY = [+0.0000, -0.0006, -0.0856, +0.0226, +1.5567, +0.0000]
PRE_GRASP     = [+0.0000, -0.7050, -0.7237, -0.1689, +1.5772, +0.0000]
GRASP         = [+0.0000, -0.8150, -0.8340, -0.1689, +1.5772, +0.0000]
```

```

def _pick(self):
    self._grip(GRIPPER_OPEN)
    self._move('home', HOME, sec=4)
    self._move('gripper_ready', GRIPPER_READY, sec=4)
    self._move('pre_grasp', PRE_GRASP, sec=4)
    self._move('grasp', GRASP, sec=5)
    self._grip(GRIPPER_CLOSED)
    self._move('grasp', GRASP, sec=2) # re-hold after close
    self._wait(1.0)
    self._grip(GRIPPER_CLOSED) # re-assert before lift
    self._wait(0.3)
    self._move('pre_grasp', PRE_GRASP, sec=5)
    self._move('home', HOME, sec=6)

```

3.6 Reach Detection Logic

```

def _at_target(self, target):
    if not self._joint_pos: return False
    for joint, t in zip(ARM_JOINTS, target):
        current = self._joint_pos.get(joint)
        if current is None: return False
        if abs(current - t) > TOLERANCE: # TOLERANCE = 0.08 rad ≈ 4.6°
            return False
    return True

```

3.7 Screenshots — Approach A

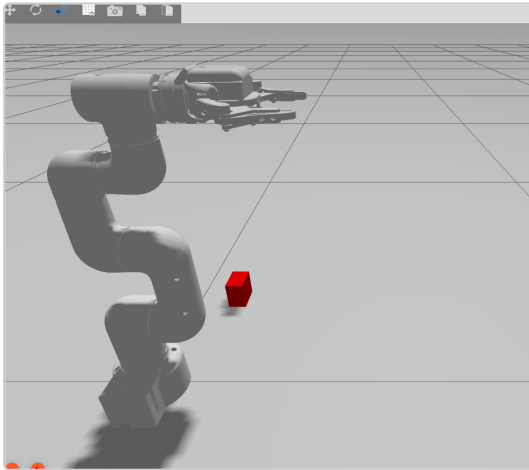


Fig. A3 — HOME position

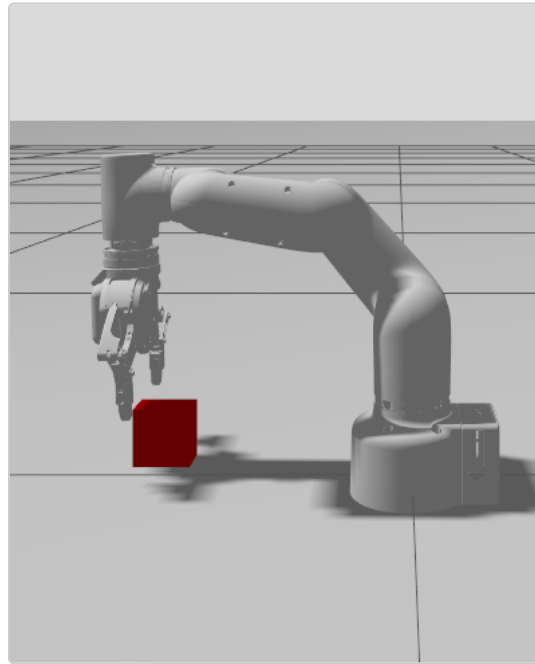


Fig. A4 — PRE_GRASP position

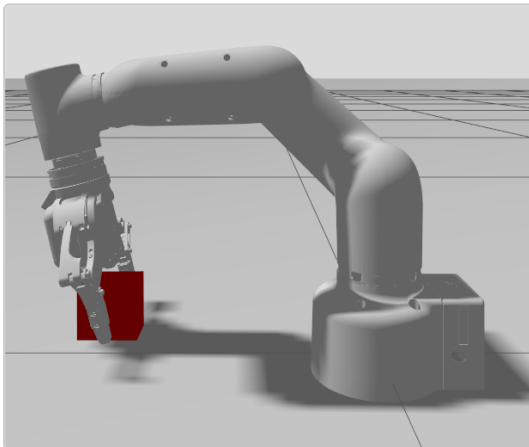


Fig. A5 — GRASP position (gripper closed)

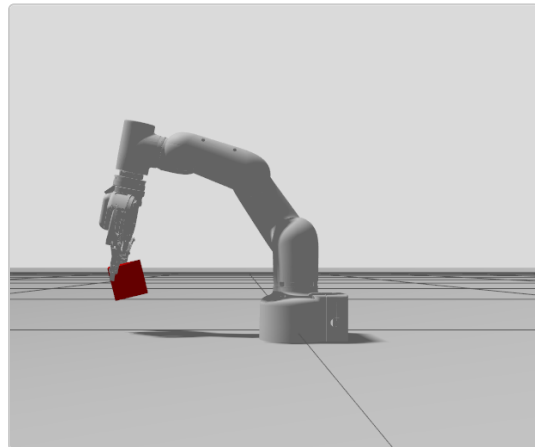


Fig. A6 — LIFT (arm returned to PRE_GRASP height)

```
Alexander@alexander-z370h-R53H:~/Projects/mycobot_ws$ source /opt/ros/humble/setup.bash && source ~/Projects/mycobot_ws/Install/setup.bash && ros2 run approach_a pick_place
[INFO] [1778792996.418849999] [pick_place]: Waiting for joint states...
[INFO] [1778792997.420873580] [pick_place]: Gripper: OPEN (0.0)
[INFO] [1778792997.522576713] [pick_place]: Moving to: home
[INFO] [1778792997.576825235] [pick_place]: Reached: home
[INFO] [1778792999.617719256] [pick_place]: Moving to: gripper_ready
[WARN] [1778793019.632787621] [pick_place]: Timeout reaching: gripper_ready - continuing anyway
[INFO] [1778793019.636315887] [pick_place]: Moving to: pre_grasp
[WARN] [1778793039.663376703] [pick_place]: Timeout reaching: pre_grasp - continuing anyway
[INFO] [1778793039.657603272] [pick_place]: Moving to: grasp
[WARN] [1778793059.672612344] [pick_place]: Timeout reaching: grasp - continuing anyway
[INFO] [1778793059.674235251] [pick_place]: Gripper: CLOSE (-1.0)
[INFO] [1778793059.777610409] [pick_place]: Moving to: grasp
[WARN] [1778793079.829111197] [pick_place]: Timeout reaching: grasp - continuing anyway
[INFO] [1778793080.834067848] [pick_place]: Gripper: CLOSE (-1.0)
[INFO] [1778793081.236090565] [pick_place]: Moving to: pre_grasp
[WARN] [1778793101.269845699] [pick_place]: Timeout reaching: pre_grasp - continuing anyway
[INFO] [1778793101.252896832] [pick_place]: Moving to: home
[INFO] [1778793101.305512090] [pick_place]: Reached: home
[INFO] [1778793103.308261275] [pick_place]: Done.
```

Fig. A7 — Terminal output of Approach A pick sequence

► **Video:** Full Approach A pick-and-place sequence — see [approaches/approach_A/screenshots/A_full_sequence.webm](#)

3.8 Advantages

- ✓ Extremely simple to implement — no math libraries required
- ✓ Fully deterministic — same angles every run
- ✓ Easy to debug — you can see exactly which joint is wrong
- ✓ No URDF parsing needed at runtime
- ✓ Works with any robot arm model

3.9 Limitations

- ✗ Every new target position requires manual re-calibration (~30-50 rqt iterations)
- ✗ Not reusable — angles are specific to the exact cube location
- ✗ No Cartesian intuition — hard to reason about XYZ space
- ✗ Cannot handle dynamic environments where objects move
- ✗ GRIPPER_READY intermediate step required for correct joint configuration

4. Approach B — Inverse Kinematics (ikpy)

4.1 Overview

In Approach B, target positions are specified in **Cartesian space** (X, Y, Z in meters). The robot's URDF kinematic chain is parsed at startup by the **ikpy** library, which numerically solves for the joint angles that place the end-effector at each target position. This eliminates the need for manual calibration.

4.2 Forward Kinematics (FK)

FK computes end-effector position from joint angles:

$$T_{06} = T_{01} \cdot T_{12} \cdot T_{23} \cdot T_{34} \cdot T_{45} \cdot T_{56}$$

where each T_{ij} is a 4x4 homogeneous transformation matrix:

$$T_{ij} = \text{Rot}(\text{axis}, \theta) \cdot \text{Trans}(dx, dy, dz)$$

Each transformation encodes the rotation (joint angle θ) and translation (link length) between consecutive coordinate frames, following Denavit-Hartenberg convention.

4.3 Inverse Kinematics (IK)

IK inverts this: given a desired end-effector position $p = [x, y, z]$, find joint angles θ :

$$\theta^* = \operatorname{argmin} \| \text{FK}(\theta) - p_{\text{target}} \|^2$$

Solved numerically by ikpy using gradient descent / L-BFGS-B optimization.

Convergence depends on the initial seed (starting guess for θ).

4.4 Tool Offset Calibration

The URDF kinematic chain ends at *link6*. The actual grasp center is ~144 mm below link6 along its local Z-axis, plus small XY offsets due to mechanical asymmetry. This offset was calibrated using Approach A GRASP angles as ground truth:

$$\text{FK}(\theta_{\text{grasp_A}}) \rightarrow T_{\text{link6_world}}$$

$$T00L_OFFSET_LOCAL = T_{\text{link6_world}}^{-1} \cdot p_{\text{cube}} - [0, 0, 0]$$

$$\text{Result: } T00L_OFFSET_LOCAL = [-0.0032, -0.0033, 0.1440] \text{ m}$$

This offset is passed to ikpy as `last_link_vector`, extending the chain.

4.5 Waypoints (Cartesian)

Waypoint	X (m)	Y (m)	Z (m)	Purpose
PRE_GRASP	0.2620	-0.0842	0.1200	90 mm above cube top
GRASP	0.2620	-0.0842	0.0450	15 mm above cube base
LIFT	0.2620	-0.0842	0.1500	120 mm above cube base

Cube position: X=0.262 m, Y=-0.0842 m, Z=0.030 m (base of cube in simulation)

4.6 IK Seed Strategy

Because IK has multiple solutions, a *seed* (initial guess) guides the solver to the correct robot configuration:

Waypoint	Seed Used	Reason
PRE_GRASP	Approach A PRE_GRASP angles	Same physical configuration desired
GRASP	Approach A GRASP angles	Ensures correct approach direction
LIFT	Solved GRASP angles	Continuity — arm stays in same config, moves only Z

Critical bug fixed: Originally LIFT used hardcoded Approach A GRASP seed. After IK solves GRASP numerically, the actual joint values differ slightly from Approach A. Using the wrong seed caused IK to find a mirror-configuration solution ($J5 \approx 2.24$ rad instead of 1.58 rad), which caused the arm to miss the cube and timeout. Fix: use the *previously solved* GRASP angles as the LIFT seed.

4.7 IK Chain Construction

```
def build_chain(urdf_path):
    chain_full = ikpy.chain.Chain.from_urdf_file(
        urdf_path,
        base_elements=['world'],
        last_link_vector=TOOL_OFFSET_LOCAL,    # [-0.0032, -0.0033, 0.144]
    )
    # Truncate at joint6output_to_joint6 to avoid gripper branch ambiguity
    stop_idx = next(i for i, l in enumerate(chain_full.links)
                    if l.name == 'joint6output_to_joint6')
    arm_links = list(chain_full.links[:stop_idx + 1]) + [chain_full.links[-1]]
    chain = ikpy.chain.Chain(arm_links)
    chain.active_links_mask = [l.name in set(ARM_JOINTS) for l in chain.links]
    return chain
```

4.8 IK Solve with Seed

```
def ik_solve(chain, arm_indices, target_xyz, seed_joints):
    seed = np.zeros(len(chain.links))
    for idx, val in zip(arm_indices, seed_joints):
        seed[idx] = val
    result = chain.inverse_kinematics(
        target_position=target_xyz,
        initial_position=seed,
        orientation_mode=None,    # position-only, no orientation constraint
    )
    joints = [result[i] for i in arm_indices]
    joints[-1] = 0.0    # force J6=0 — keeps gripper perpendicular
    return joints
```

4.9 Pick Sequence

```
def _pick_sequence(self):
    self._open_gripper()
    self._move_joints('home', HOME_JOINTS, sec=4)

    solved = {}
    seeds = {'pre_grasp': SEED_PRE_GRASP, 'grasp': SEED_GRASP}

    for name in ['pre_grasp', 'grasp', 'lift']:
        seed = solved.get('grasp', seeds.get(name, SEED_GRASP))
        joints = ik_solve(self._chain, self._arm_indices,
                          WAYPOINTS_CART[name], seed)
        solved[name] = joints
        if name == 'grasp':
            self._move_joints(name, joints, sec=5, dwell=2.0)
            self._close_gripper(joints)  # 6 s gripper close, arm held fixed
        elif name == 'lift':
            self._move_joints(name, joints, sec=3, dwell=0.5)
        else:
            self._move_joints(name, joints, sec=5, dwell=0.5)

    self._move_joints('home', HOME_JOINTS, sec=4)
```

4.10 Screenshots — Approach B

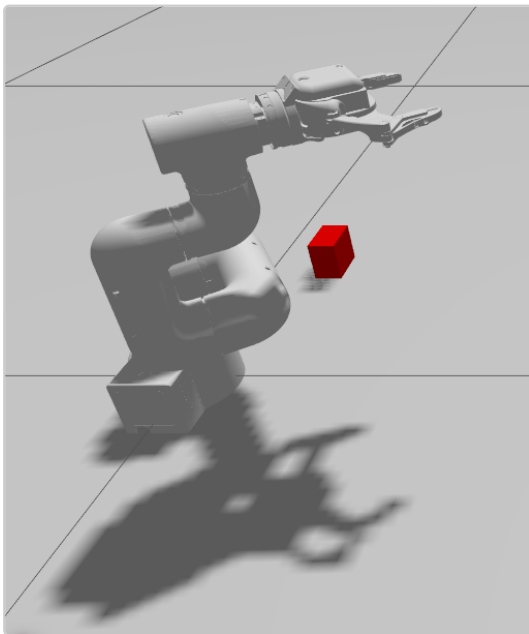


Fig. B1 — HOME position (auto-set at launch)

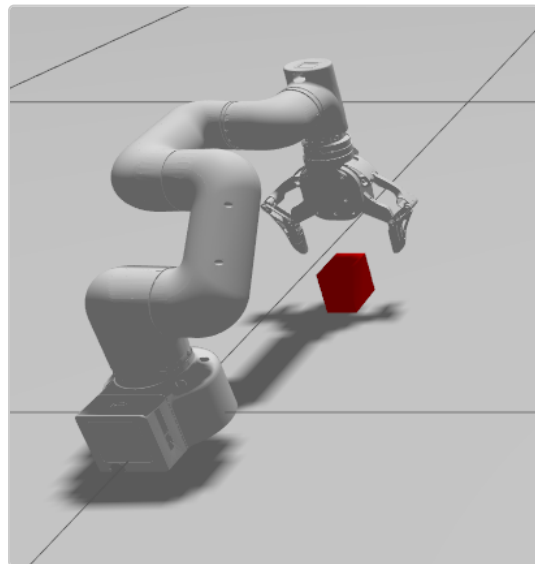


Fig. B2 — PRE_GRASP (IK solved: $X=0.262$, $Y=-0.084$, $Z=0.120$)

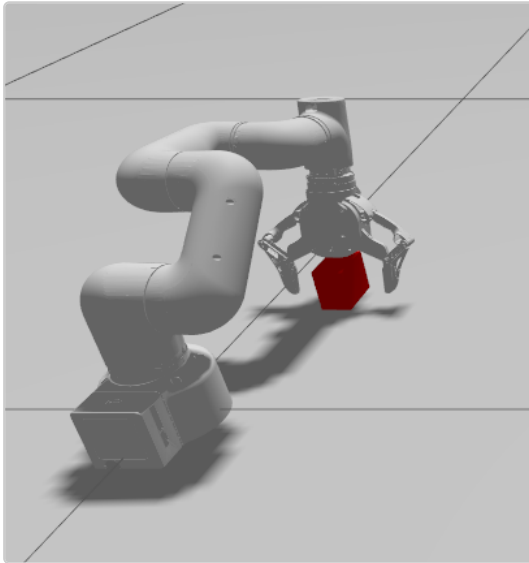


Fig. B3 — GRASP position (IK solved: $Z=0.045$)

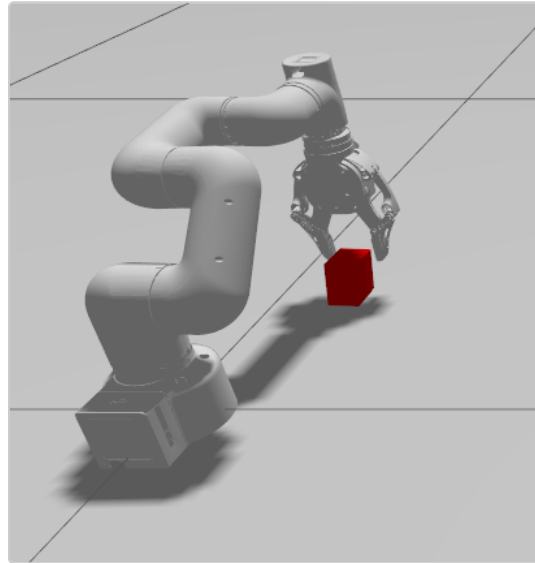


Fig. B4 — LIFT (IK solved: $Z=0.150$)

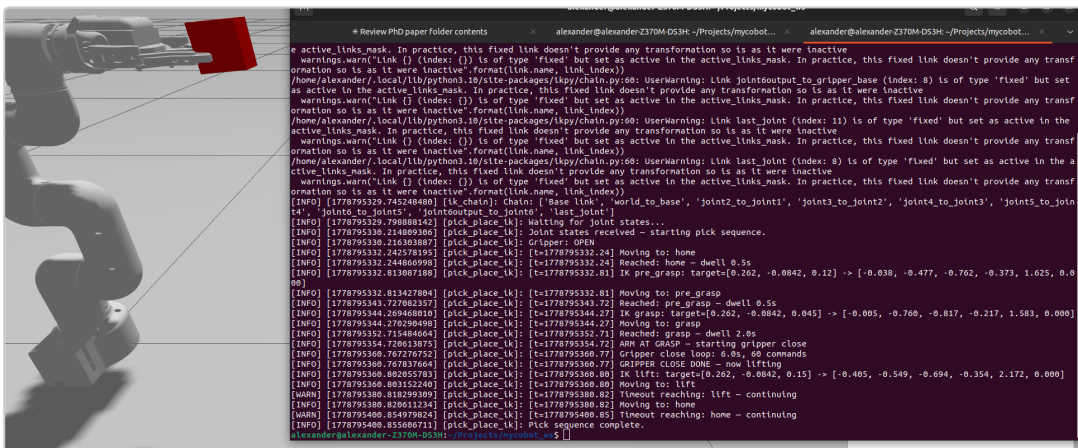


Fig. B5 — Terminal output showing IK-solved joint angles and sequence log

► **Video:** Full Approach B pick-and-place sequence — see [approaches/approach_B/screenshots/B_full_sequence.webm](#)

4.11 Advantages

- ✓ Waypoints specified in intuitive Cartesian (XYZ) coordinates
- ✓ New targets require only changing XYZ constants — no rqt iteration
- ✓ Extensible: can add more waypoints trivially
- ✓ Foundation for sensor-driven grasping (camera → XYZ → IK)
- ✓ Automatic home position at launch via set_home_pose node

4.12 Limitations

- ✗ Requires URDF and ikpy dependency
- ✗ IK has multiple solutions — wrong seed causes wrong configuration
- ✗ Tool offset must be calibrated from known reference
- ✗ Position-only IK: no orientation constraint (gripper angle not controlled)
- ✗ Same Gazebo physics limitation: gripper slip under dynamic lift

5. Comparison

Criterion	Approach A (Hardcoded)	Approach B (IK)
Waypoint specification	Joint angles [rad]	Cartesian XYZ [m]
Setup effort	30–50 rqt iterations per pose	Set XYZ once, IK computed automatically
Math required	None (visual alignment)	FK/IK, homogeneous transforms
Dependencies	rclpy, trajectory_msgs	+ ikpy, numpy, URDF
Determinism	100% same each run	Deterministic if seed fixed
New target pose	Re-calibrate manually	Change 3 numbers (X,Y,Z)
Gripper orientation	Controlled via J5 angle	Not constrained (orientation_mode=None)
Code complexity	Simple (~100 lines)	Moderate (~250 lines)
Intermediate poses needed	Yes (GRIPPER_READY)	No — IK handles configuration
Scalability	Poor — manual per pose	Good — one IK call per pose
Sensor integration readiness	No	Yes (XYZ from camera)
Gripper hold (simulation)	Limited (physics)	Limited (same physics)

6. Known Issues and Fixes

Issue	Approach	Fix Applied
Arm not at home on launch	B	Added set_home_pose node triggered after arm_controller loads
IK wrong solution for LIFT ($J_5 \approx 2.24$)	B	Use solved GRASP angles as LIFT seed instead of hardcoded values
Controllers fail on second launch	Both	Kill all ROS2/Gazebo processes before relaunch
Duplicate package name conflict	Both	COLCON_IGNORE in paper/ directory
Gripper slip during lift	Both	Documented as Gazebo physics limitation; not fully solvable without grasp plugin
Arm does not reach grasp depth	A	Iterative rqt calibration: J2 adjusted from -0.705 to -0.815 rad

7. Running the Code

Approach A

```
# Build
colcon build --packages-select approach_a --symlink-install
source install/setup.bash

# Launch simulation (auto-homes arm)
ros2 launch approach_a gz_harmonic.launch.py

# Run pick-and-place (in new terminal)
ros2 run approach_a pick_place
```

Approach B

```
# Install IK dependency
pip install ikpy

# Build
colcon build --packages-select pick_place_ik --symlink-install
source install/setup.bash

# Launch simulation (auto-homes arm)
ros2 launch pick_place_ik gz_harmonic_ik.launch.py
```

```
# Run pick-and-place (in new terminal)
ros2 run pick_place_ik pick_place_ik
```

8. Conclusion

Both approaches successfully demonstrate pick-and-place manipulation of a target object in the Gazebo Harmonic simulation. **Approach A** is the recommended starting point for beginners: it requires no mathematics, is easy to debug, and produces reliable results once calibrated. However, it does not scale — every new pick location requires a full manual re-calibration cycle.

Approach B eliminates this scalability problem by working in Cartesian space. A new target requires only changing three coordinate values. This is the natural foundation for sensor-driven manipulation where a camera provides object XYZ positions at runtime. The additional complexity (URDF parsing, IK library, seed management, tool offset calibration) is justified for any application with more than one pick location or for dynamic environments.

Both approaches share the same fundamental simulation limitation: Gazebo's contact model does not provide sufficient friction to reliably hold the cube during dynamic arm acceleration. A production-quality solution would require either a grasp-attachment plugin (e.g., gz-sim GripperPlugin), a higher-fidelity physics engine, or a PID-controlled gripper with force feedback.

9. References

1. Elephant Robotics. *mycobot_320 M5 Technical Specification*. 2022.
2. Open Robotics. *ROS2 Humble Documentation*. docs.ros.org, 2022.
3. Open Robotics. *ros2_control: JointTrajectoryController*. GitHub, 2022.
4. Mayer, B. *ikpy: Inverse Kinematics for Python*. GitHub: Phylliade/ikpy, 2016.
5. Open Robotics. *Gazebo Harmonic (gz-sim 8) Documentation*. gazebosim.org, 2023.
6. Denavit, J., Hartenberg, R.S. *A kinematic notation for lower-pair mechanisms*. ASME Journal, 1955.